

Lenguajes de Programación III

Trabajo Practico 2.

Prof.: Ing. Gustavo Sosa Cataldo

Planteamiento del T. P.:

Este trabajo involucra el desarrollo de una implementación utilizando conceptos vistos en clase y laboratorio sobre Posix Threads (Pthreads).

La implementación corresponde al 50% del parcial.

Luego de la corrección en clase, habrá un desarrollo adicional a realizarlo allí en el momento que valdrá el 50% restante.

Máxima cantidad de integrantes por grupo: 4

Todos los integrantes del grupo deben estar **presentes el día de la entrega/defensa**, el que no esté presente lleva ausente.

La fecha de entrega/defensa es el lunes 1 de junio (es decir el día del primer parcial) a las 15:00 horas.

Gestión de Procesos - PThreads

Sistema Concurrente de Atención de Pedidos

Objetivo

Implementar un sistema concurrente utilizando POSIX Threads que simule el procesamiento de pedidos de un restaurante/delivery, aplicando sincronización mediante mutex y semáforos.

Descripción General

Desarrollar un programa en C llamado `delivery_system.c` que simule el funcionamiento interno de un sistema de preparación de pedidos.

El sistema tendrá:

- Un conjunto de threads **productores** que generan pedidos
- Un conjunto de threads **cocineros** que preparan pedidos
- Un conjunto de threads **repartidores** que entregan pedidos

Todos los pedidos deberán pasar obligatoriamente por las siguientes etapas:

Generación → Preparación → Entrega

Funcionamiento

1) Generadores de pedidos

- Existen **N** threads productores.
- Cada productor genera pedidos periódicamente.
- Cada pedido posee:
 - ID único
 - Tiempo estimado de preparación
 - Tipo de comida

Los pedidos se insertan en una cola compartida de “pedidos pendientes”.

2) Cocineros

- Existen **M** threads cocineros.
 - Cada cocinero:
 - Toma un pedido pendiente
 - Simula preparación usando **sleep()**
 - Mueve el pedido a la cola de “listos para entrega”
-

3) Repartidores

- Existen **R** threads repartidores.
 - Cada repartidor:
 - Retira un pedido listo
 - Simula entrega
 - Marca el pedido como entregado
-

Requisitos de Sincronización

El sistema debe implementar correctamente:

Mutex

Utilizar **pthread_mutex_t** para proteger:

- Colas compartidas
- Contadores globales
- IDs de pedidos

Semáforos

Utilizar semáforos POSIX (**sem_t**) para:

- Controlar cantidad de pedidos disponibles
 - Controlar capacidad máxima de las colas
 - Coordinar disponibilidad entre productores/consumidores
-

Requisitos Técnicos Obligatorios

El trabajo debe utilizar obligatoriamente:

- `pthread_create`
 - `pthread_join`
 - `pthread_mutex_lock`
 - `pthread_mutex_unlock`
 - `sem_init`
 - `sem_wait`
 - `sem_post`
 - `sem_destroy`
-

Requisitos Funcionales

El programa deberá:

- Permitir configurar por constantes:
 - Cantidad de productores
 - Cantidad de cocineros
 - Cantidad de repartidores
 - Tamaño máximo de colas
- Mostrar mensajes de eventos:
 - Pedido generado
 - Pedido tomado por cocinero
 - Pedido entregado
- Evitar:
 - Race conditions
 - Deadlocks
 - Busy waiting